

UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO
EMILIA

Dipartimento di Ingegneria “Enzo Ferrari”

Corso di laurea in Ingegneria Informatica

CROSS-COMPILAZIONE DI PROGETTI JAVAFX SU
DISPOSITIVI ANDROID

Relatore:
Prof. Nicola Bicocchi

Candidato:
Matteo Sissa

Anno Accademico 2022-2023

Indice

1. Sommario

2. Introduzione a JavaFX

2.1 Libreria grafica

2.2 Ciclo di vita di un'applicazione JavaFX

2.3 Costruzione dell'interfaccia grafica con FXML

2.4 Look and Feel con CSS

3. Tecnologie ausiliarie

3.1 Maven

3.2 GRAALVM

3.3 Gluon Substrate

3.4 GitHub Actions

4. Cross-compilazione di un progetto JavaFX su dispositivi Android

5. Conclusioni

6. Bibliografia / Sitografia

Capitolo 1

Sommario

Il presente documento è stato redatto con lo scopo di presentare una tecnologia software abbastanza recente sviluppata da Oracle, ovvero JavaFX, e le sue notevoli potenzialità per cross-compilare il codice sorgente su diverse piattaforme target, nello specifico su dispositivi Android.

La cross-compilazione può essere definita come un processo di compilazione compiuto da una macchina per generare del codice eseguibile su un sistema o piattaforma diverso dal proprio. In altre parole, il codice sorgente di un'applicazione viene cross-compilato quando viene reso eseguibile per un sistema diverso da quello su cui la compilazione è avvenuta.

JavaFX consente lo sviluppo di desktop e web applications in maniera strutturata ed efficiente, e presenta anche il supporto per cross-compilare su svariate piattaforme, tra le quali sistemi operativi desktop (Windows, MacOS, Linux), ma anche dispositivi mobile (Android, iOS) e sistemi embedded (Raspberry Pi).

Il documento è organizzato nel seguente modo. Nella prima sezione di questo elaborato si esploreranno le interessanti potenzialità di JavaFX, le sue principali caratteristiche e modi di utilizzo.

Nel successivo capitolo si elencheranno le tecnologie ausiliarie necessarie per portare a termine il processo di cross-compilazione, con una breve descrizione del loro scopo e ruolo nel progetto.

Infine, la parte fondamentale del documento che verterà su una descrizione dettagliata dei passaggi richiesti per cross-compilare il codice sorgente JavaFX su dispositivi Android.

Capitolo 2

Introduzione a JavaFX

JavaFX può essere definita come una piattaforma software rilasciata per la prima volta da Oracle nel 2008. L'ultima release, nel momento in cui il documento viene redatto, è del 2022. Si può quindi considerare una tecnologia estremamente recente, sviluppata all'interno del mondo Java. JavaFX mira a rendere la creazione di applicazioni client e di GUI molto agevole per il programmatore, infatti era nato come progetto per sostituire il framework Swing in Java. Questo primo intento è stato poi abbandonato, ma le librerie JavaFX sono state incluse nell'OpenJDK a partire da JDK 11.

2.1 Libreria grafica

Partendo dall'aspetto puramente grafico, JavaFX mette a disposizione una vasta gamma di classi e interfacce per la creazione di GUI anche molto complesse. Tra gli elementi fondamentali ci sono layout di vario tipo per l'organizzazione spaziale dei componenti, un'ampia selezione di GUI controls (Button, Label, TextField...) e moltissimi tipi di eventi supportati per l'interattività con l'utente.

Qui di seguito (figura 1.1.) viene riportato uno schema grafico della struttura generale di una interfaccia grafica standard in JavaFX:

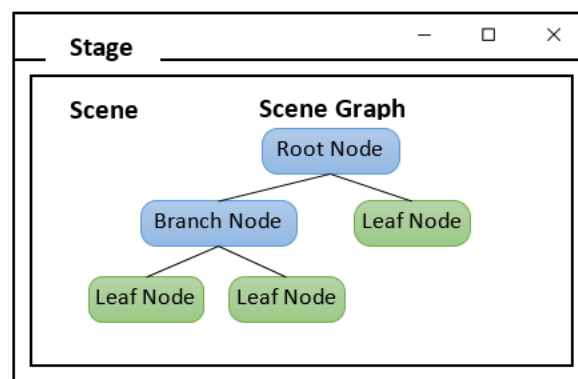


Figura 1.1.

Il ruolo di “top-level container” (contenitore di livello più elevato) è assegnato alla classe Stage, la quale mostra solamente il pannello di controllo in alto, con i bottoni standard per la gestione di una finestra (chiusura, ridimensionamento a icona e schermo intero). Un’applicazione può avere uno o più Stage (scorrelati o interdipendenti tra loro).

Allo Stage si aggiunge un’istanza della classe Scene, la quale ospita il “Scene graph”, ovvero la struttura gerarchica di tutti i componenti grafici che popolano l’interfaccia. La Scene deve avere associato un componente “Root”, il quale solitamente è in grado di “ospitare” e occuparsi del layout di altri componenti, in una struttura ad albero come quella in figura.

Si rimanda all’API per un elenco completo di tutti i GUI controls disponibili.

2.2 Ciclo di vita di un’applicazione JavaFX

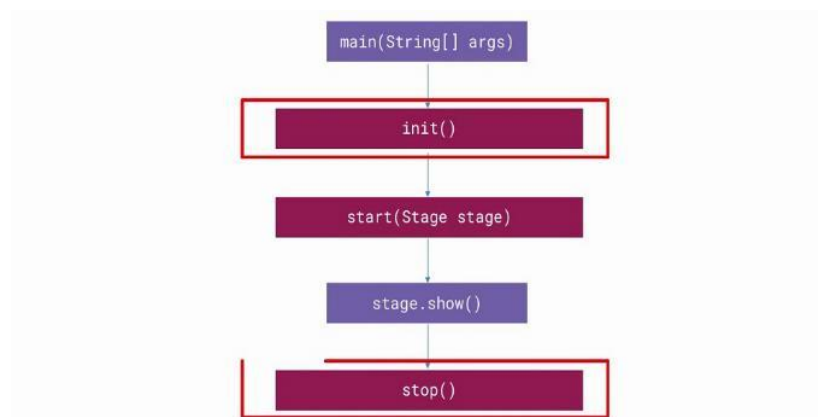


Figura 1.2.

L’API (Application Programming Interface) di JavaFX contiene una classe chiamata Application, che è anche l’entry point di una qualsiasi applicazione sviluppata con questo framework. Quando il JavaFX Runtime Environment comincia l’esecuzione di un programma, crea un’istanza della classe Application presente tra le classi del progetto, e comincia a eseguire in un ordine specifico alcuni dei metodi contenuti in questa classe, più precisamente:

- Invoca il metodo `init ()`
- Chiama il metodo `start (Stage stage)`
- Aspetta che l'applicazione termini
- Infine, invoca il metodo `stop ()`

Questo processo è fondamentale per capire come poter cominciare a scrivere un programma JavaFX. È necessaria una classe che funga da entry point e quindi che estenda `Application` ed esegua l'overriding dei suoi metodi.

Dopodiché, il metodo `start (Stage stage)` è quello indicato per contenere la parte più sostanziale di codice che consente all'applicazione di funzionare.

Il JavaFX Runtime Environment istanzierà quella classe al momento dell'esecuzione e chiamerà i metodi nell'ordine descritto.

JavaFX introduce anche degli elementi estremamente innovativi per lo sviluppo di interfacce grafiche, che migliorano la qualità del codice e semplificano il lavoro del programmatore, con un conseguente aumento dell'efficienza nel processo di sviluppo.

È grazie ad alcuni di questi strumenti che la costruzione di un'applicazione client anche complessa viene scomposta e separata in obiettivi più piccoli, con lo scopo di ridurre la difficoltà. I prossimi due punti vogliono illustrare queste funzionalità.

2.3 Costruzione dell'interfaccia grafica con FXML

FXML è un linguaggio basato su una sintassi XML che permette di descrivere in maniera precisa l'intero layout che l'interfaccia grafica dell'applicazione JavaFX deve mostrare. Sostanzialmente, è possibile in questo modo generare un file “.fxml” che contenga la descrizione della disposizione di tutti i componenti grafici e le loro caratteristiche, separando totalmente questo aspetto dal codice logico dell'applicazione.

Questo modo di programmare presenta notevoli vantaggi. Per prima cosa, il codice risulta molto più pulito e leggibile, dovendosi occupare solamente degli aspetti logici e non grafici o stilistici.

Ciò implica anche che il testing dell'interfaccia grafica viene svolto molto più agevolmente, in quanto eventuali errori sono limitati al file “.fxml” dove è semplice identificarli e correggerli.

Inoltre, FXML non è un linguaggio compilato, di conseguenza le modifiche apportate a questo file non richiedono una ricompilazione del codice per poterle visivamente vedere.

Legato a questo aspetto innovativo di JavaFX, l'applicazione Scene Builder è stata sviluppata per rendere la scrittura del file FXML ancora più immediata. Scene Builder è uno strumento di design che consente di visualizzare l'interfaccia grafica in fase di progettazione, permettendo all'utente di progettare l'interfaccia grafica in maniera interattiva e senza scrivere alcun tipo di codice.

Una volta ottenuto il risultato desiderato, Scene Builder si occupa di generare automaticamente il file FXML da includere nella cartella del progetto.

2.4 Look and Feel con CSS

Lo stesso approccio appena descritto per la costruzione dell'interfaccia grafica viene adottato anche per programmare e modificare lo stile dei componenti grafici, il cosiddetto “look and feel”.

CSS (“Cascading Style Sheet”) è un linguaggio utilizzato moltissimo nel mondo web per descrivere l'aspetto dei singoli elementi di una pagina web. JavaFX riprende lo stesso concetto, permettendo di raggruppare in un file “.css” tutte le specifiche relative allo stile dei componenti sull'interfaccia (colori, sfondi, sfumature, font...).

Questa separazione degli aspetti più grafici dal codice logico rende il progetto molto più semplice da gestire, da testare e chiaro da leggere e comprendere.

Capitolo 3

Tecnologie ausiliarie

Per poter far funzionare correttamente il processo di cross-compilazione del codice sorgente JavaFX su piattaforme target differenti, è necessario far collaborare e interagire diverse tecnologie e piattaforme software che svolgono specifici ruoli.

Questa sezione ha lo scopo di illustrare quali sono queste tecnologie, mostrare ad alto livello il loro funzionamento e dare una visione generale di cosa succede internamente quando il codice sorgente JavaFX viene cross-compilato.

3.1 Maven

Maven può essere definito come un “build automation tool”, ovvero uno strumento di automazione per gestire il building o creazione di un progetto software, che è composto da diversi processi, tra cui compilazione del codice sorgente per renderlo codice binario, packaging del codice binario, esecuzione dei test ...

È uno strumento utilizzato principalmente per progetti Java, per cui particolarmente adatto anche per JavaFX. Per configurare un progetto Maven è necessario un file con il nome “pom.xml” all’interno della home directory del progetto. Questo file deve contenere dei comandi strutturati con una sintassi XML.

Il primo grande obiettivo per cui Maven viene impiegato nella realizzazione di un progetto è per la risoluzione delle dipendenze del progetto. Una dipendenza è un software o una libreria esterna al proprio codice le cui funzionalità sono utilizzate dall’applicazione in fase di sviluppo.

Senza Maven, lo sviluppatore sarebbe costretto a eseguire manualmente il download di tutte le librerie e componenti software necessari a far funzionare correttamente il proprio

progetto e a inserire tali componenti in opportune cartelle in modo da renderle accessibili all'applicazione.

Maven consente di dichiarare le dipendenze del proprio codice all'interno del file “pom.xml” (con una specifica sintassi) e sarà il building tool stesso a eseguire il download dinamico di tali componenti in una directory locale quando necessario.

Il repository di default in cui le librerie vengono ricercate è “Maven 2 Central Repository”, ma è possibile configurarne altri sempre all'interno del file “pom.xml”. La sintassi per le dipendenze segue lo schema sotto riportato:

```
<dependencies>
  <dependency>
    <!-- coordinates of the required library -->
    <groupId>junit</groupId>
    <version>3.8.1</version>
  </dependency>
</dependencies>
```

In questo esempio si è aggiunta una dipendenza per la libreria di testing JUnit, versione 3.8.1.

Definite correttamente tutte le dipendenze, Maven mette a disposizione diversi “goal”, ovvero obiettivi che possono essere richiesti sul progetto. Questi sono fondamentalmente dei comandi che riguardano aspetti di building o testing dell'applicazione. Tra i più comunemente usati ci sono:

- compile: per compilare il codice sorgente in codice binario
- test: per eseguire i test sul codice in produzione
- package: raggruppa il codice compilato in un formato distribuibile, per esempio file JAR
- deploy: copia il risultato del processo package in un repository remoto

Si può notare come Maven si prenda cura di tutti gli aspetti fondamentali del processo di costruzione di un'applicazione software. Per eseguire uno di questi obiettivi, si può utilizzare Maven come strumento a linea di comando. Ad esempio, è possibile

posizionarsi da terminale sulla cartella home del progetto (dove si trova il file “pom.xml”) e digitare:

```
mvn compile
mvn package
```

Per compilare e costruire il package.

Quelli appena presentati sono gli obiettivi di base che vengono forniti dalla versione standard di Maven, ma il secondo grande vantaggio di questo strumento è costituito dai plug-ins che si possono aggiungere per ampliarne ulteriormente le capacità.

I plug-ins consentono di eseguire le stesse operazioni coinvolte nel building di un’applicazione (compile, test, package...) ma declinate sulla base del plug-in scelto e in maniera da renderle adattabili alle specifiche esigenze di un progetto.

Per esempio, è possibile inserire un plug-in per eseguire e fare il building di codice JavaFX. La sintassi nel file “pom.xml” è la seguente:

```
<plugins>
  <plugin>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-maven-plugin</artifactId>
    <version>${javafx.plugin.version}</version>
  </plugin>
</plugins>
```

Una volta dichiarato correttamente un plug-in, è possibile utilizzarne le funzionalità sempre da linea di comando con la sintassi:

```
mvn [nome-plugin]:[nome-goal]
```

Ad esempio:

```
mvn javafx:run
```

Per eseguire il codice JavaFX.

In questo caso, il building tool risolverà tutte le dipendenze del codice e poi eseguirà il goal “javafx:run” sulla base di come lo specifico plug-in dichiarato nel file “pom.xml” è stato configurato e costruito.

La maggior parte degli IDE consente di avere Maven direttamente integrato all’interno dell’ambiente di sviluppo, per cui l’esecuzione di questi comandi diventa ancora più immediata con un pannello dedicato, quindi senza bisogno di aprire il terminale.

3.2 GRAALVM

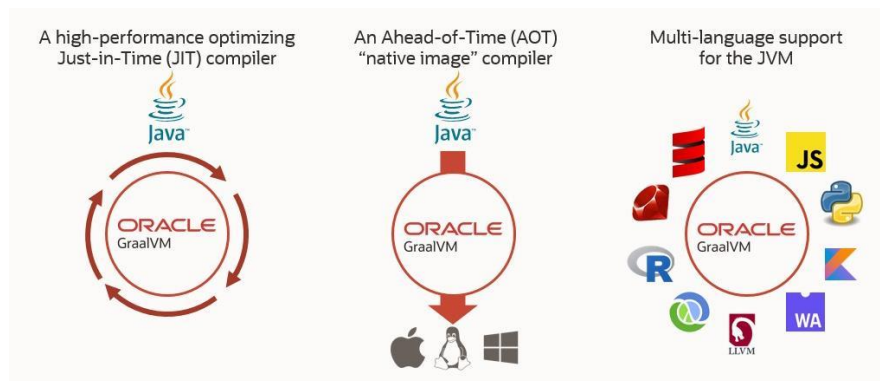


Figura 1.3.

GraalVM è un JDK altamente performante in grado di velocizzare le prestazioni delle applicazioni Java eseguite su una JVM. Questo strumento presenta diverse innovazioni, tra le quali l'utilizzo di un compilatore JIT (just-in-time) alternativo (per migliorare le performances dell'applicazione), il fatto di essere multilingua, cioè di avere la possibilità di compilare codice sorgente scritto in vari linguaggi di programmazione, e la tecnologia "native image" o immagine nativa, ovvero una funzionalità di compilazione del codice Java in anticipo, o AOT (ahead-of-time).

Per quanto riguarda l'impiego di questa tecnologia nella discussione di questa tesi, è necessario solamente comprendere la tecnologia "native image", indispensabile per far funzionare il processo di cross-compilazione del codice JavaFX verso dispositivi mobile.

Partendo dalle basi, bisogna avere chiaro come il codice sorgente Java viene compilato e poi eseguito su un qualsiasi sistema. Da sempre, lo slogan legato al linguaggio Java è "Write once, run everywhere", che significa "Scrivi (il codice) una volta, esegilo ovunque".

Uno dei principali obiettivi di questo linguaggio è proprio quello di non necessitare di compilare il codice sorgente in svariati eseguibili di diverso tipo per ogni sistema target, bensì quello di offrire un processo di compilazione uniforme.

Proprio a questo scopo, è stato necessario introdurre un ulteriore strato software, ovvero la Java Virtual Machine, per poter attuare a runtime il processo di trasformazione del codice agnostico (o platform-independent) in codice macchina comprensibile dalla piattaforma target su cui il software deve girare.

Il processo di compilazione che viene quindi svolto per programmi Java si compone di una prima compilazione parziale del codice sorgente in Java bytecode. Quest'ultimo può essere pensato come una rappresentazione intermedia tra il codice sorgente e il codice binario di una qualsiasi macchina target.

La grande novità sta nel fatto che il bytecode è universale, perché è compreso da qualsiasi JVM. Di conseguenza, può essere impacchettato in formati come JAR e distribuito. Quando un utente vuole eseguire tale programma, deve avere installata la specifica JVM che esegue la traduzione del bytecode nel codice macchina del proprio sistema.

Questa fase è una seconda fase compilativa e nella maggior parte delle JVM standard viene eseguita con un compilatore JIT (just-in-time), ovvero la traduzione in codice macchina del bytecode avviene esattamente prima dell'esecuzione stessa del programma (cioè a runtime).

In sostanza, l'utente esegue l'applicazione Java (ad esempio impacchettata in un file JAR), un'istanza della JVM viene creata come processo utente, il bytecode del programma è fornito al compilatore JIT della JVM che lo traduce in linguaggio macchina e il risultato finale del processo di compilazione è un codice binario comprensibile dalla CPU specifica del sistema host.

Questo meccanismo è estremamente efficiente per sistemi desktop (con sistemi operativi come Linux, MacOS, Linux), ma non si sposa molto bene con l'ambiente mobile ed embedded. I motivi sono molteplici, e dipendono dal caso che si prende in considerazione.

Nel mondo Android, per esempio, il bytecode Java contenuto in file con formato JAR non è eseguibile. Android sfrutta un bytecode differente (proprietario) che viene compilato su una macchina virtuale chiamata Dalvik, differente dalle standard JVM.

Nei sistemi iOS, invece, è l'esecuzione stessa di una Java VM il problema, perché Apple non supporta JVM per iOS; quindi, il codice Java non può essere eseguito a partire dal bytecode su iPhones o iPads.

Questo è il motivo per cui la funzionalità di compilatore AOT (ahead-of-time) fornita da GraalVM è fondamentale per completare il processo di cross-compilazione di codice sorgente JavaFX verso dispositivi mobile.

Un compilatore viene definito AOT quando trasforma il codice sorgente in codice macchina prima dell'esecuzione stessa del programma sul sistema target, cioè prima del runtime. Compilazione ed esecuzione vengono così separate in due momenti diversi.

La tecnologia “native image” di GraalVM parte da codice sorgente Java (JavaFX nel nostro caso) e, per prima cosa, identifica tutte le librerie e classi dell'API Java o JavaFX con cui l'applicazione stessa che deve essere compilata ha delle dipendenze.

A questo punto, esegue la compilazione di tutto il codice (sia il codice sorgente applicativo, sia le librerie Java necessarie) in codice macchina specifico per una data piattaforma target.

Il risultato è un file binario contenente un eseguibile autonomo (senza alcuna dipendenza esterna) che può essere direttamente fornito al sistema target per l'esecuzione.

Nel caso presentato in questo documento, GraalVM sarà in grado di tradurre direttamente il codice Java (JavaFX) in un eseguibile “.apk” che contiene il codice macchina direttamente computabile da un dispositivo Android.

In questo modo, il dispositivo mobile non ha alcun bisogno di istanziare una JVM a runtime.

3.3 Gluon Substrate

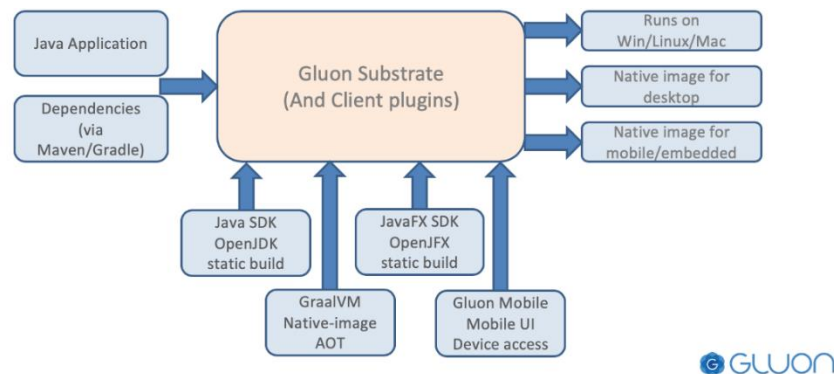


Figura 1.4.

Arrivati a questo punto della discussione, risulta abbastanza chiaro che gli estremi del processo di cross-compilazione sono stati ben definiti. Da una parte, l'applicazione è stata sviluppata in JavaFX e questo corrisponde al codice sorgente.

Dall'altro lato, GraalVM consente di compilare il codice in Java bytecode per rendere l'applicazione eseguibile sui sistemi dotati di una JVM, oppure direttamente in codice macchina specifico per la piattaforma target, grazie al compilatore AOT e alla tecnologia "native image", nel caso di sistemi mobile o embedded.

Rimane solamente da comprendere come sia possibile far correttamente collaborare e interagire le varie tecnologie coinvolte. A questo scopo serve il Gluon Substrate, ovvero uno strato software aggiuntivo in grado di semplificare il compito di mettere insieme tutti i pezzi e farli interfacciare senza errori.

In figura 1.4 viene riportato uno schema a blocchi riassuntivo che mostra il ruolo del Gluon Substrate nel processo di cross-compilazione.

Si possono facilmente identificare tutti gli elementi citati ed elencati precedentemente in questo documento. Alla sinistra è presente il codice sorgente, costituito dall'applicazione JavaFX e da tutte le dipendenze evidenziate da Maven per librerie esterne. In basso, sono raffigurate le librerie delle API Java e Java FX su cui l'applicazione si appoggia per poter funzionare, nonché GraalVM, con il compilatore AOT e la tecnologia "native image". Al

centro, il Gluon Substrate “accoglie” tutti gli elementi citati come input e produce i file eseguibili elencati sulla destra.

La particolarità che rende il Gluon Substrate molto fruibile è il fatto che è stato reso disponibile anche come un plug-in di Maven. Di conseguenza, basta modificare leggermente il file “pom.xml” per estendere le funzionalità del tool di building con le capacità e i servizi offerti dal Gluon Substrate.

Come si vedrà nel seguito della discussione, è quindi possibile configurare il file pom.xml per rendere disponibili comandi come:

```
mvn gluonfx:build  
mvn gluonfx:compile  
mvn gluonfx:package
```

Che rendono tutte le capacità di building di un progetto software che sono implementate nel Gluon Substrate direttamente fruibili attraverso Maven.

3.4 GitHub Actions

GitHub è un sistema di hosting per software principalmente open source, diventato ormai indispensabile per la gestione di grandi progetti e soprattutto per consentire la collaborazione di molti programmatori sullo stesso codice.

Offre la possibilità di avere un repository remoto dove salvare le modifiche apportate all'applicazione in fase di sviluppo e consente a chiunque stia cooperando al progetto di scaricare tali modifiche in locale e progredire ulteriormente a partire da esse.

Oltre a questi servizi, a partire dal 2018 è stata introdotta una nuova feature con il nome di GitHub Actions. Fondamentalmente, lo scopo delle GitHub Actions è quello di automatizzare completamente il processo di building e deployment di un'applicazione ospitata sulla piattaforma.

La grande potenzialità di questo servizio è rinforzata anche dal fatto che GitHub mette a disposizione delle macchine virtuali remote che ospitano i principali sistemi operativi desktop (Ubuntu Linux, Windows, MacOS).

Tutti i processi di compilazione, esecuzione, testing, packaging, building e deployment per l'applicazione in fase di sviluppo su GitHub possono essere automatizzati all'interno di una delle macchine virtuali a scelta e il modo in cui questo deve avvenire è proprio specificato dal programmatore attraverso le Actions.

Per essere più precisi, le Action costituiscono l'elemento unitario di un intero workflow, il quale non è altro se non un file con sintassi YAML che contiene l'elenco di tutti gli step e i processi da portare a termine nella macchina virtuale remota scelta.

Un singolo progetto software può contenere più di un singolo workflow e tutti i workflow devono essere raggruppati nella cartella “.github/workflow” (percorso relativo alla home del progetto).

Elenchiamo adesso alcuni dei termini ed elementi fondamentali relativi a questa tecnologia, che saranno poi importanti per capire il processo di cross-compilazione presentato nel prossimo capitolo.

Per prima cosa, un workflow deve essere associato ad uno o più eventi che provocano l'esecuzione stessa del workflow.

Ad esempio, un workflow potrebbe essere utilizzato tutte le volte che vengono eseguite operazioni di push o pull sul repository GitHub, oppure si può configurare l'esecuzione manuale.

Il “runner” è la tipologia di macchina virtuale remota che deve ospitare il programma e le varie operazioni che vengono eseguite su di esso. Vengono messe a disposizione macchine virtuali Ubuntu Linux, Windows e MacOS.

I “jobs” costituiscono un elenco di steps all'interno del workflow che viene eseguito sullo stesso runner. Un workflow può contenere più jobs che verranno completati concorrentemente su macchine virtuali differenti.

Infine, le “actions” sono delle piccole applicazioni sviluppate sulla tecnologia GitHub Actions che permettono di semplificare lo svolgimento di operazioni spesso ripetitive e

molto frequenti. Consentono di ridurre il codice da scrivere all'interno di un workflow, facendo leva su codice di terze parti.

Ad esempio, le actions consentono di automatizzare il download di software di vario tipo sulle macchine virtuali, oppure salvare determinati file generati all'interno della macchina virtuale nel repository di GitHub dove l'applicazione risiede.

Tutti gli elementi citati possono essere specificati e utilizzati con sintassi apposite all'interno del file YAML che descrive un workflow. Nel capitolo successivo si vedranno solamente i dettagli degli aspetti necessari per comprendere il processo di cross-compilazione che verrà presentato.

Il motivo per il quale si è scelto di usare i workflows e le GitHub Actions in questa tesi è per rendere il processo di cross-compilazione verso dispositivi Android il più generale possibile e replicabile in maniera esatta da qualsiasi lettore.

Al momento in cui il documento viene elaborato, la tecnologia Gluon Substrate supporta la cross-compilazione verso Android solo se i vari processi necessari vengono ospitati all'interno di un sistema Linux.

L'utilizzo di macchine virtuali remote rende questa limitazione non bloccante per chiunque non disponga nell'immediato di tale sistema operativo in locale.

Capitolo 4

Cross-compilazione di un progetto JavaFX su dispositivi Android



Figura 1.5.

Per permettere la cross-compilazione di un progetto JavaFX è necessario e sufficiente mettere insieme tutte le tecnologie descritte nei capitoli precedenti. Il presente documento si concentra sulla cross-compilazione verso dispositivi Android.

Per rendere la trattazione più semplice, si suppone di stare sviluppando il progetto all'interno di un IDE, qui si fa riferimento a IntelliJ IDEA, ma tutti gli altri ambienti di sviluppo lavorano più o meno allo stesso modo.

Il primo passo è quello di creare il codice sorgente, ovvero un'applicazione JavaFX. Deve quindi essere definita una classe che estenda `Application` e che funga da “entry point” per l'esecuzione come spiegato nel primo capitolo. Con questo punto di partenza, è possibile sbizzarrirsi con la realizzazione di un qualsiasi progetto applicativo. L'interfaccia grafica può essere strutturata con l'aiuto di FXML e CSS per rendere lo sviluppo più semplice e agevole.

Allo scopo di dimostrare il funzionamento del processo qui descritto, è stata creata un'applicazione demo che fungerà da prova per verificare i vari step. Si tratta di un semplice gioco in cui è possibile generare un numero arbitrario di sfere di vari colori che vengono fatte muovere in maniera casuale sullo schermo. Il tutto è controllato dall'utente attraverso tre bottoni. In figura 1.5 viene riportata un'immagine rappresentativa di come l'applicazione si mostra su un sistema Windows.

Il bottone “start” consente di far muovere le sfere sullo schermo, “add” aggiunge una nuova sfera e “stop” ferma tutti gli elementi.

Per poter eseguire correttamente l'applicazione, visto che l'interfaccia grafica è sviluppata con codice JavaFX, è necessario risolvere le dipendenze con le librerie JavaFX, e a questo ci pensa Maven e il file “pom.xml”.

Una copia completa del file viene mostrata alla fine del documento per questioni di maggiore leggibilità e chiarezza espositiva; sono riportati alcuni frammenti di codice durante le spiegazioni per illustrare gli aspetti fondamentali.

Maven risolve automaticamente le dipendenze con l'API di JavaFX, semplicemente aggiungendo le seguenti linee di codice:

```
<dependency>
  <groupId>org.openjfx</groupId>
  <artifactId>javafx-controls</artifactId>
  <version>${javafx.version}</version>
</dependency>
```

Si può qui facilmente notare la sintassi per la risoluzione delle dipendenze evidenziata nel capitolo 3.1.

L'altro pezzo fondamentale è il Gluon Substrate, che può essere introdotto nel progetto come plug-in di Maven. In altre parole, si modifica ancora una volta il file “pom.xml” in modo tale da far acquisire al tool Maven tutte le capacità di building del Gluon Substrate illustrate nel capitolo 3.3. Questo è il frammento di codice relativo al plug-in:

```
<plugin>
  <groupId>com.gluonhq</groupId>
  <artifactId>gluonfx-maven-plugin</artifactId>
  <version>${gluonfx.plugin.version}</version>
  <configuration>
    <target>${gluonfx.target}</target>
```

```
<attachList>
    <list>display</list>
    <list>lifecycle</list>
    <list>statusbar</list>
    <list>storage</list>
</attachList>
<mainClass>${mainClassName}</mainClass>
</configuration>
</plugin>
```

Ci sono inoltre alcune dipendenze da aggiungere relative sempre al Gluon Substrate che è possibile vedere a fine documento nel file “pom.xml” completo.

Ciò che conta ai fini della comprensione è che grazie a questo plug-in, vengono sbloccati nuovi goal o obiettivi del comando Maven con cui è facilmente possibile accedere alle funzionalità di building del Gluon Substrate. Per esempio, è possibile usare i comandi:

```
mvn gluonfx:compile
mvn gluonfx:build
mvn gluonfx:run
```

Per compilare, buildare o eseguire il programma facendo leva sulle funzionalità del Gluon Substrate.

Con questi comandi, il processo di compilazione e building del codice su un sistema Desktop è fondamentalmente concluso. Difatti, se ci si posiziona da terminale sulla cartella home del progetto e si esegue uno dei comandi sopra indicati, Maven automaticamente deriverà la piattaforma software su cui il comando viene lanciato (Windows, Linux, MacOS) e sarà in grado di portare a termine l’obiettivo richiesto (compilazione, building o esecuzione).

La parte più complicata, in cui entra in gioco anche tutta la tecnologia di GraalVM, è dovuta al servizio di cross-compilazione dello stesso codice JavaFX su dispositivi mobile. In questa tesi ci si concentrerà sul mondo Android. Per raggiungere questo obiettivo, è necessario introdurre nell’insieme di strumenti in utilizzo anche GraalVM e le GitHub Actions.

È importante ricordarsi, infatti, che il supporto per la cross-compilazione verso Android è attualmente disponibile con il Gluon Substrate solamente all’interno di un sistema Linux. Per rendere la trattazione del tutto generale e replicabile dal lettore, è stato fatto uso delle macchine virtuali remote messe a disposizione dalla piattaforma GitHub.

Come prima cosa, è necessario avere un repository GitHub collegato al progetto che si vuole cross-compilare. Tutti i file, le classi, e il file “pom.xml” devono essere sincronizzati sul repository GitHub.

Se si sta seguendo il processo da un IDE come IntelliJ IDEA, è possibile generare il repository remoto direttamente all’interno dell’ambiente di sviluppo, in modo molto pratico e veloce.

È poi necessario eseguire il caricamento di tutto il progetto attraverso un’operazione di “push”.

Per poter controllare il processo di building che verrà messo in atto sulla macchina virtuale Linux, si deve progettare il file YAML che descrive il workflow da eseguire. Questo file deve essere inserito all’interno della cartella “.github/workflows” (percorso relativo dalla home del progetto).

Per la spiegazione qui riportata, si fa riferimento ad un workflow funzionante che porta a termine tutto il processo di cross-compilazione per Android, e se ne si spiegano gli aspetti fondamentali. Nel seguito della spiegazione vengono ripresi alcuni frammenti per illustrarne il funzionamento, mentre il file completo viene riportato in fondo al documento per consentire maggiore leggibilità.

Per prima cosa viene specificato in base a quale evento il workflow deve essere eseguito, con la keyword “on”:

```
on: workflow_dispatch
```

In questo caso, viene impostata un’esecuzione manuale.

Subito dopo, comincia l’unico job dell’intero workflow, ovvero la serie di step sequenziali che devono essere portati a termine su una stessa macchina virtuale.

Il job viene chiamato “build” e comincia specificando la versione di macchina virtuale su cui deve essere eseguito, ovvero l’ultima versione di Ubuntu.

```
runs-on: ubuntu-latest
```

Segue una GitHub Action, ovvero:

```
# Checkout your project
- uses: actions/checkout@v2
```

Le Actions sono piccole applicazioni o programmi scritti apposta per semplificare la stesura dei workflows, rendendo immediate operazioni che si ripresentano

frequentemente. In questo caso, l'action "checkout" è necessaria per copiare l'intero progetto che si sta cross-compilando all'interno della macchina virtuale. Di default, infatti, GitHub non esegue questa operazione automaticamente.

A questo punto, è necessario verificare che la macchina virtuale abbia scaricato l'ultima versione di GraalVM, elemento fondamentale per compilare codice JavaFX verso dispositivi mobile. Per fare ciò, si utilizza nuovamente un'Action, sviluppata da Gluon, ovvero:

```
- name: Setup GraalVM built by Gluon
  uses: gluonhq/setup-graalvm@master
```

"setup-graalvm@master" esegue il download dell'ultima versione di GraalVM e imposta anche le variabili d'ambiente del sistema Linux in modo tale da consentire al Gluon Substrate di rintracciare tale tecnologia quando sarà necessaria nel processo di building.

È tutto pronto per lanciare la cross-compilazione del codice con target Android, questo viene fatto con il seguente frammento di YAML:

```
- name: Gluon Build
  run: |
    export ANDROID_SDK=$ANDROID_HOME
    mvn -Pandroid gluonfx:build gluonfx:package
```

La parte fondamentale è il comando da terminale mvn (l'ultima riga del frammento di codice). L'opzione "-P" consente di dichiarare la piattaforma target, per cui Android nel caso qui descritto. I goal che vengono richiesti sono il building e il packaging per generare il file eseguibile, ovvero "gluonfx:build" e "gluonfx:package".

Questo è il punto in cui tutti gli strumenti software descritti in questo documento arrivano a cooperare per raggiungere l'obiettivo finale della cross-compilazione. Per comprendere cosa stia realmente succedendo quando questo comando viene lanciato, bisogna ricordarsi che il comando mvn si basa sul contenuto del file "pom.xml", che è esattamente quello incluso nel progetto di partenza.

Quindi, il codice sorgente è scritto in JavaFX, e dentro al file "pom.xml" il modo per risolvere tutte le dipendenze con le librerie di JavaFX è definito, per cui l'accesso all'API di JavaFX è garantita.

Inoltre, le capacità di building di Maven vengono potenziate e ampliate dal plug-in del Gluon Substrate, ancora una volta settato e configurato nel file “pom.xml”. Il Gluon Substrate viene invocato proprio grazie ai comandi da terminale “gluonfx:build” e “gluonfx:package”.

Visto che è specificata la piattaforma target Android, il Gluon Substrate è programmato per fare leva sui servizi offerti da GraalVM per generare un’immagine nativa dell’applicazione per dispositivi Android. Di conseguenza, ricercherà la posizione del software di GraalVM, che viene identificata grazie alle variabili d’ambiente definite nello step precedente del workflow.

GraalVM partirà a questo punto dal codice sorgente JavaFX, deriverà tutte le dipendenze esterne con le librerie Java e JavaFX, compilerà quindi tutto il codice per la piattaforma Android. Mettendo insieme i pezzi, il Gluon Substrate completa il building del progetto per Android e genera il file “.apk”.

Gli ultimi comandi eseguiti nel file YAML servono solamente per creare una nuova cartella all’interno della macchina virtuale remota, trasferire al suo interno il file “.apk” e poi fare l’upload di tale nel repository GitHub, per renderlo disponibile all’utente a processo concluso.

Quando il workflow ha terminato, aprendo il proprio progetto su GitHub, nella sezione “Actions”, selezionando il workflow corretto, è possibile vedere tra gli “Artifacts” una cartella compressa scaricabile che contiene l’eseguibile “.apk”.

Per dimostrare il funzionamento del processo appena descritto, il gioco di prova che è stato presentato all'inizio del capitolo è stato cross-compilato e il file “.apk” installato in un dispositivo HUAWEI P20 con versione 9 di Android.

Il risultato è soddisfacente. Viene riportato uno screenshot eseguito direttamente dal cellulare:

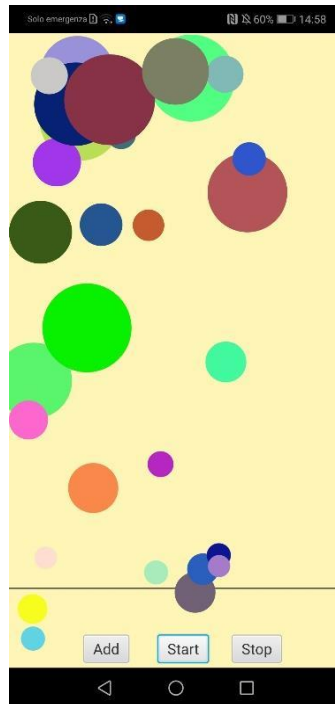


Figura 1.6.

L'installazione di un file “.apk” su un dispositivo Android è relativamente semplice. È sufficiente trasferire l'eseguibile sul dispositivo mobile (ad esempio da computer con un collegamento USB) e poi cliccare sull'”.apk” direttamente dallo schermo del cellulare. Una finestra di installazione si aprirà automaticamente.

Capitolo 5

Conclusioni

La possibilità di cross- compilare uno stesso codice sorgente per diverse piattaforme target è un obiettivo estremamente rilevante nel mondo dello sviluppo software attuale, perché consente di astrarre dalle moltissime tecnologie e sistemi hardware su cui uno stesso progetto applicativo può essere eseguito.

Nel mondo Java in particolare, si è sempre pensato all'impiego della JVM come soluzione per rendere il codice adattabile a qualsiasi piattaforma, ma questa opzione risulta inefficiente per sistemi mobile ed embedded.

Di conseguenza, un'applicazione scritta in Java o JavaFX per sistemi Desktop dovrebbe essere riscritta con altri linguaggi (seppur simili) per poterla adattare a sistemi Android o iOS, rendendo il processo di sviluppo molto più lento e gravoso.

Questo è il motivo per cui tecnologie come JavaFX, GraalVM e il Gluon Substrate costituiscono un grande passo avanti per il mondo Java e per la sua versatilità.

Le capacità compilative di GraalVM, unite alle grandi innovazioni introdotte da JavaFX nello sviluppo di applicazioni Java client sempre più ricche e dettagliate da un punto di vista grafico, il tutto fatto collaborare grazie al Gluon Substrate, rendono l'intero servizio estremamente avanzato e proiettato al futuro.

Questa tesi mirava proprio a testare le capacità delle tecnologie elencate, a dimostrarne l'utilizzo pratico e a evidenziarne i punti di forza.

Il processo di documentazione e configurazione di tutti gli elementi in gioco non si può ancora definire semplice e immediato, difatti richiede diverse competenze e tentativi per riuscire a far cooperare tutti gli strumenti software in maniera corretta. Nonostante ciò, i risultati ottenuti da questo studio si possono considerare più che soddisfacenti, visto che la struttura funziona e porta a termine con successo il compito per cui è stata ideata.

Bibliografia / Sitografia

[1] JavaFX API

[<https://openjfx.io/javadoc/19/index.html>]

[2] JavaFX Oracle tutorials

[<https://docs.oracle.com/javase/8/javase-clienttechnologies.htm>]

[3] FXML documentation

[https://docs.oracle.com/javafx/2/fxml_get_started/jfxpub-fxml_get_started.pdf]

[4] Maven Wikipedia webpage

[https://en.wikipedia.org/wiki/Apache_Maven]

[5] Maven documentation

[<https://maven.apache.org/guides/>]

[6] Presentazione all'Oracle Code Rome 2019 su GraalVM

[[GraalVM: Run Programs Faster Everywhere](#)]

[7] GraalVM Oracle Page

[<https://www.oracle.com/it/java/graalvm/what-is-graalvm/>]

[8] Gluon documentation

[<https://docs.gluonhq.com/>]

[9] Gluon Substrate block diagram

[<https://gluonhq.com/gluon-substrate-and-graalvm-native-image-with-javafx-support/>]

[10] GitHub Actions documentation

[<https://docs.github.com/en/actions>]

File pom.xml completo

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.ballgame</groupId>
    <artifactId>ballgame</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>
    <name>BallGame</name>

    <properties>
        <project.build.sourceEncoding>UTF-
8</project.build.sourceEncoding>
        <maven.compiler.release>11</maven.compiler.release>
        <javafx.version>20</javafx.version>
        <attach.version>4.0.18</attach.version>
        <gluonfx.plugin.version>1.0.18</gluonfx.plugin.version>
        <javafx.plugin.version>0.0.8</javafx.plugin.version>
        <mainClassName>com.ballgame.BallMain</mainClassName>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.openjfx</groupId>
            <artifactId>javafx-controls</artifactId>
            <version>${javafx.version}</version>
        </dependency>
        <dependency>
            <groupId>com.gluonhq</groupId>
            <artifactId>charm-glisten</artifactId>
            <version>6.2.3</version>
        </dependency>
        <dependency>
            <groupId>com.gluonhq.attach</groupId>
            <artifactId>display</artifactId>
            <version>${attach.version}</version>
        </dependency>
        <dependency>
            <groupId>com.gluonhq.attach</groupId>
            <artifactId>lifecycle</artifactId>
            <version>${attach.version}</version>
        </dependency>
    </dependencies>
</project>
```

```

    <dependency>
      <groupId>com.gluonhq.attach</groupId>
      <artifactId>statusbar</artifactId>
      <version>${attach.version}</version>
    </dependency>
    <dependency>
      <groupId>com.gluonhq.attach</groupId>
      <artifactId>storage</artifactId>
      <version>${attach.version}</version>
    </dependency>
    <dependency>
      <groupId>com.gluonhq.attach</groupId>
      <artifactId>util</artifactId>
      <version>${attach.version}</version>
    </dependency>
  </dependencies>

```

```

  <repositories>
    <repository>
      <id>Gluon</id>

```

```

    <url>https://nexus.gluonhq.com/nexus/content/repositories/releases</url>

```

```

    </repository>
  </repositories>

```

```

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.8.1</version>
      </plugin>

      <plugin>
        <groupId>org.openjfx</groupId>
        <artifactId>javafx-maven-plugin</artifactId>
        <version>${javafx.plugin.version}</version>
        <configuration>
          <mainClass>${mainClassName}</mainClass>
        </configuration>
      </plugin>

      <plugin>
        <groupId>com.gluonhq</groupId>
        <artifactId>gluonfx-maven-plugin</artifactId>
        <version>${gluonfx.plugin.version}</version>
        <configuration>
          <target>${gluonfx.target}</target>
          <attachList>

```

```
        <list>display</list>
        <list>lifecycle</list>
        <list>statusbar</list>
        <list>storage</list>
    </attachList>
    <mainClass>${mainClassName}</mainClass>
</configuration>
</plugin>
</plugins>
</build>

<profiles>
    <profile>
        <id>android</id>
        <properties>
            <gluonfx.target>android</gluonfx.target>
        </properties>
    </profile>
</profiles>
</project>
```

File YAML completo (GitHub workflow):

```
on: workflow_dispatch

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      # Checkout your project
      - uses: actions/checkout@v2

      - name: Setup GraalVM built by Gluon
        uses: gluonhq/setup-graalvm@master
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }

      # Install extra requirements on top of ubuntu-latest
      - name: Install libraries
        run: |
          sudo apt-get update
          sudo apt install libasound2-dev libavcodec-dev
libavformat-dev libavutil-dev libgl-dev libgtk-3-dev
libpangol.0-dev libxtst-dev

      - name: Gluon Build
        run: |
          export ANDROID_SDK=$ANDROID_HOME
          mvn -Pandroid gluonfx:build gluonfx:package

      # Create staging directory in which the apk will be copied
      - name: Make staging directory
        run: mkdir staging

      # Copy the apk to the staging directory
      - name: Copy native images to staging
        run: cp -r target/gluonfx/aarch64-android/gvm/*apk
staging

      # Upload the staging directoy as a build artifact
      - name: Upload
        uses: actions/upload-artifact@v2
        with:
          name: Package
          path: staging
```